



COMPTE-RENDU

STM32

réalisé par : Adam IBRHOUNTANE, Simon DARTIGUELONGUE

Fait le : 21/05:2026

1. Introduction

Dans le cadre du module R2.07 d'informatique embarquée, plusieurs travaux dirigés et travaux pratiques ont été réalisés afin de découvrir l'environnement de développement des microcontrôleurs STM32 ainsi que les principaux périphériques matériels intégrés à ces systèmes embarqués.

Les différentes séances avaient pour objectif de se familiariser progressivement avec :

- la configuration du microcontrôleur à l'aide de STM32CubeMX,
- la programmation en langage C sous l'environnement Keil,
- ainsi que l'utilisation de la bibliothèque HAL fournie par STMicroelectronics.

Au cours des différents TP, plusieurs fonctionnalités fondamentales des systèmes embarqués ont été étudiées et mises en œuvre, notamment :

- les entrées/sorties numériques GPIO,
- les interruptions externes,
- la génération de signaux PWM,
- les communications SPI et I2C,
- la conversion analogique-numérique (ADC),
- ainsi que la communication série UART.

Ces différents travaux ont ensuite été regroupés dans une application unique permettant de piloter et superviser un moteur à courant continu à l'aide de plusieurs périphériques connectés au microcontrôleur STM32F334R8Tx. Le système développé permet notamment :

- de contrôler le sens de rotation du moteur,
- de faire varier sa vitesse grâce à un signal PWM,
- d'afficher différentes informations sur un écran OLED,
- de visualiser le niveau de commande grâce à un vu-mètre à LEDs,
- et de communiquer avec un ordinateur via une liaison série UART.

Ce compte rendu présente donc l'ensemble des manipulations réalisées au cours des TP ainsi que l'architecture matérielle et logicielle mise en œuvre pour le développement du projet final.

2. Présentation du système

Carte microcontrôleur utilisée :

Le projet a été développé autour du microcontrôleur STM32F334R8Tx une carte développée au département GEII par Martial Leyney et Damien Blanchard pour le pilotage d'un moteur à courant continu.. Ce microcontrôleur possède plusieurs périphériques matériels intégrés permettant la réalisation de systèmes embarqués complets.

La programmation du microcontrôleur a été réalisée en langage C à l'aide :

- du logiciel STM32CubeMX pour la configuration matérielle,
- de l'environnement Keil µVision pour le développement et la compilation du programme.

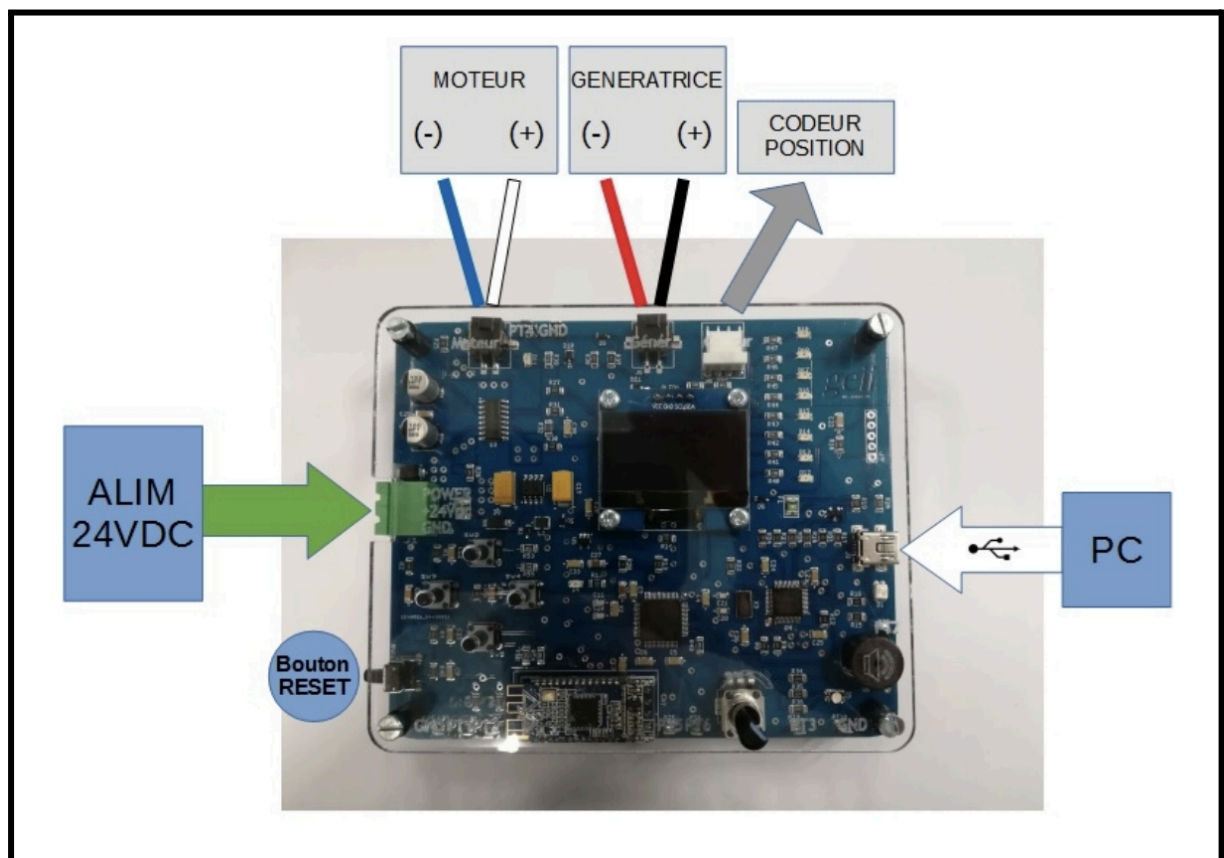


Figure 1 : Présentation de la carte STM32 :

Explication du montage :

Ce montage sert à commander et observer le fonctionnement d'un moteur à courant continu (MCC) 24 V à l'aide d'une carte STM32.

- La carte STM32 génère un signal PWM (signal carré visible à l'oscilloscope) pour contrôler la vitesse du moteur.
- Le moteur MCC reçoit cette commande et tourne plus ou moins vite selon le rapport cyclique du PWM.
- L'alimentation continue (Laboratory DC Power Supply) fournit l'énergie au montage et au moteur, ici avec une tension réglée autour de 24 V max.
- L'oscilloscope WaveAce 101 permet de visualiser le signal PWM envoyé par la STM32 afin de vérifier sa fréquence et son rapport cyclique.
- Les sondes et câbles relient les différents appareils pour mesurer les tensions et commander le moteur.

En résumé : la STM32 pilote le moteur grâce à un signal PWM, l'alimentation fournit l'énergie, et l'oscilloscope permet de contrôler la qualité du signal de commande.
(Voir figure 2)

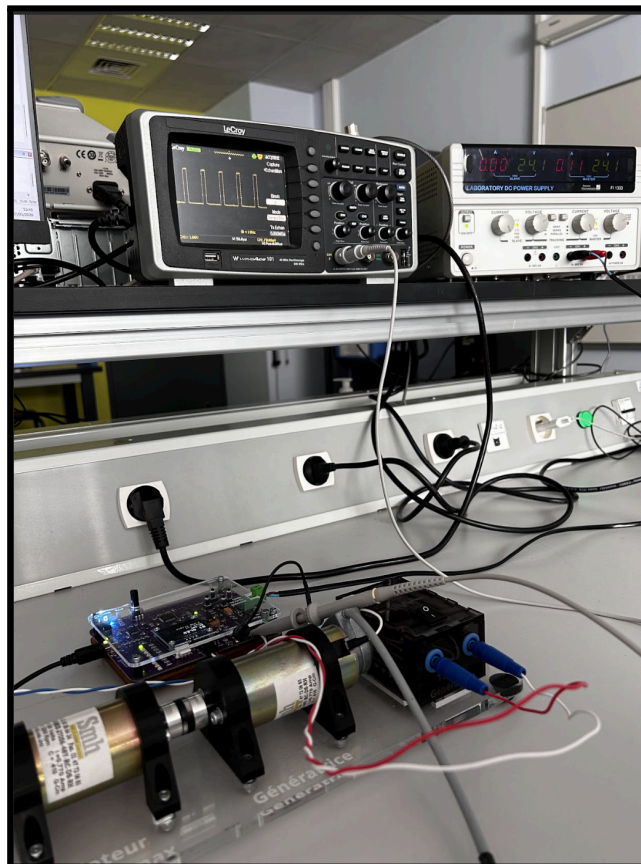


Figure 2 : Montage

Configuration des périphériques :

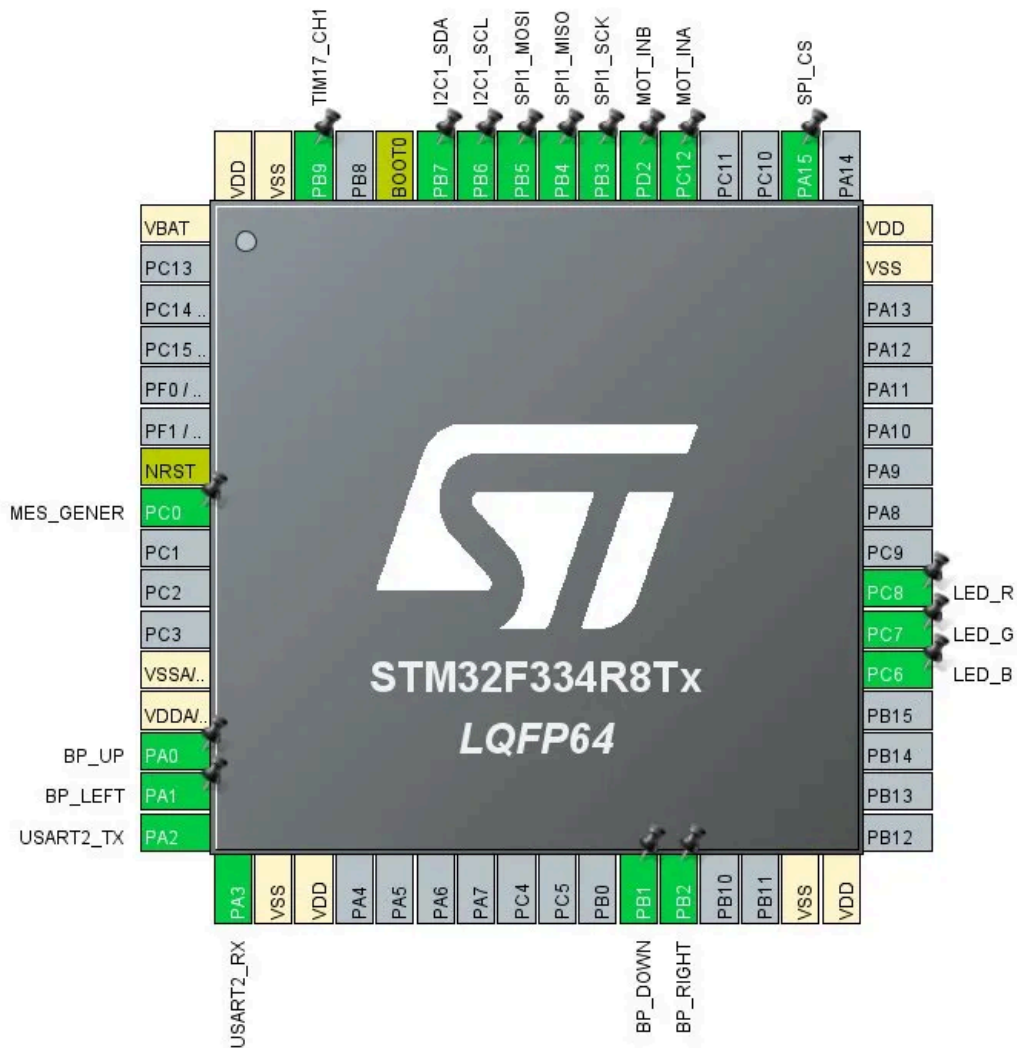


Figure : Configuration et nomenclature des pins

Entrées utilisateur — GPIO Input avec interruptions externes (EXTI) Les quatre boutons poussoirs sont connectés aux broches PA0 (BP_UP), PA1 (BP_LEFT), PB1 (BP_DOWN) et PB2 (BP_RIGHT). Ces broches sont configurées en entrées numériques (GPIO Input). De plus, les boutons UP et DOWN sont utilisés avec interruptions externes (EXTI), ce qui permet au microcontrôleur de réagir instantanément à chaque appui sans avoir à interroger continuellement l'état des broches. Chaque bouton est associé à une action de contrôle du moteur : augmentation ou diminution de la vitesse, et changement du sens de rotation.

Affichage RGB — GPIO Output Les broches PC6 (LED_B), PC7 (LED_G) et PC8 (LED_R) sont configurées en sorties numériques (GPIO Output) et pilotent les trois composantes d'une LED RGB. En combinant ces trois canaux, il est possible de reproduire différentes couleurs afin d'indiquer visuellement l'état du système, comme le sens de rotation ou le niveau de vitesse du moteur.

Commande moteur — GPIO Output et PWM (TIM17) La commande du moteur repose sur deux types de signaux. Les broches PC12 (MOT_INA) et PD2 (MOT_INB) sont configurées en sorties numériques (GPIO Output) et contrôlent le pont en H en définissant le sens de rotation du moteur. La broche PB9 (TIM17_CH1) est quant à elle configurée en sortie PWM via le timer TIM17, et génère un signal modulé en largeur d'impulsion permettant de régler finement la vitesse du moteur.

Communication SPI — Vu-mètre LEDs (MCP23S08) Les broches PB3 (SPI1_SCK), PB4 (SPI1_MISO) et PB5 (SPI1_MOSI) sont configurées en fonction alternative SPI1 et assurent respectivement l'horloge, la réception et l'émission des données sur le bus SPI. La broche PA15 (SPI_CS) est configurée en sortie numérique (GPIO Output) et sert de signal Chip Select pour activer le circuit MCP23S08, qui pilote le vu-mètre à LEDs.

Communication I2C — Écran OLED (SSD1306) Les broches PB6 (I2C1_SCL) et PB7 (I2C1_SDA) sont configurées en fonction alternative I2C1 et permettent la communication avec l'écran OLED SSD1306. SCL transporte le signal d'horloge tandis que SDA transporte les données, permettant l'affichage d'informations textuelles ou graphiques sur l'écran.

Conversion analogique — ADC1 La broche PC0 (MES_GENER) est configurée en entrée analogique sur le canal ADC1. Elle mesure la tension générée par le moteur en mode générateur, ce qui permet d'estimer la vitesse de rotation de manière indirecte sans capteur dédié.

Communication série — USART2 Les broches PA2 (USART2_TX) et PA3 (USART2_RX) sont configurées en fonction alternative USART2 et assurent une liaison série bidirectionnelle avec un ordinateur hôte. Cette interface est utilisée pour le débogage, la supervision ou l'envoi de commandes depuis un terminal série.

3. Travaux Pratiques

TP N°1 Pilotage de Leds à partir de boutons poussoirs :

Objectif :

- L'objectif de ce premier TP était de piloter les LED RGB en mode tout ou rien à l'aide des boutons poussoirs. À travers cet exercice, nous avons pu prendre en main la maquette ainsi que les outils de développement du microcontrôleur STM32, en réalisant trois programmes simples permettant de comprendre le fonctionnement des entrées/sorties numériques et l'interaction entre les boutons et les LED.

Rappel du cahier des charges :

→ Dans ce TP, les consignes du TD1 demandaient de configurer les GPIO du STM32 avec CubeMX puis de développer plusieurs programmes permettant de contrôler les LED RGB à partir des boutons poussoirs. Le travail consistait notamment à gérer l'allumage et l'extinction des LED, à utiliser la fonction HAL_GPIO_ReadPin pour lire l'état des boutons, ainsi qu'à implémenter des interruptions afin d'éviter le mode polling permanent.

Algorithme :

pour répondre à cela, on a s'est d'abord occupé de l'initialisation :

- Configurer les broches des LED en sortie.
- Configurer les boutons poussoirs en entrée.
- Éteindre les trois LED au démarrage du programme.

Ensuite Gestion du bouton BP_DOWN avec interruption :

Attendre un appui sur le bouton BP_DOWN.

Lorsqu'un appui est détecté :

- inverser l'état de la LED verte :
- si elle est éteinte → l'allumer ;
- si elle est allumée → l'éteindre.

Retourner en attente d'un nouvel appui.

→ L'avantage des interruptions est qu'elles permettent au microcontrôleur de réagir immédiatement à l'appui sur le bouton sans devoir vérifier en permanence son état dans la boucle principale.

Ensuite, Gestion du bouton BP_RIGHT :

Vérifier en permanence l'état du bouton BP_RIGHT.

Si le bouton est appuyé :

- changer l'état d'une variable de clignotement.

Si le mode clignotement est activé :

- allumer puis éteindre la LED bleue toutes les 500 ms.

Sinon :

- garder la LED éteinte.

Code

→ Voici le code réalisée (Pour rappel vous trouverez ci-joint Le fichier PDF qui contient tout le code de ce TP TP1, et en annexe le fichier contenant l'entièreté du code réalisé pendant ce projet avec les commentaires ajoutés)

Comme demandé, les trois leds seront éteintes au démarrage :

```
/*_USER_CODE BEGIN 2 */
HAL_GPIO_WritePin (LED_B_GPIO_Port,LED_B_Pin, GPIO_PIN_SET) ;
HAL_GPIO_WritePin (LED_G_GPIO_Port,LED_G_Pin, GPIO_PIN_SET) ;
HAL_GPIO_WritePin (LED_R_GPIO_Port,LED_R_Pin, GPIO_PIN_SET) ;

/* USER CODE END 2 */
```

Par la suite, Dans la boucle on utilise la structure vu dans le TD N°1 afin de réaliser le programme

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    // bouton gauche allumé : led bleue allumée(relâche bouton : éteinte)
    if (HAL_GPIO_ReadPin(BP_LEFT_GPIO_Port, BP_LEFT_Pin) == GPIO_PIN_RESET) {
        HAL_GPIO_WritePin (LED_B_GPIO_Port,LED_B_Pin, GPIO_PIN_RESET) ; }
    else {
        HAL_GPIO_WritePin (LED_B_GPIO_Port,LED_B_Pin, GPIO_PIN_SET) ;
    }
    // bouton droit enfoncé : change l'état du drapeau clignote_red

    if (HAL_GPIO_ReadPin(BP_RIGHT_GPIO_Port,BP_RIGHT_Pin) == GPIO_PIN_RESET) {

        clignote_red = !clignote_red;
        while(HAL_GPIO_ReadPin(BP_RIGHT_GPIO_Port,BP_RIGHT_Pin) == GPIO_PIN_RESET);

    }
    if(clignote_red) {
        HAL_GPIO_TogglePin(LED_R_GPIO_Port,LED_R_Pin);
        HAL_Delay(500);
    }
    else {
        HAL_GPIO_WritePin(LED_R_GPIO_Port,LED_R_Pin, GPIO_PIN_SET);
    }
}

/* USER CODE END WHILE */
```


TP N°2 Commande en boucle ouverte du moteur à courant continu :

Objectif :

- Ce deuxième travail pratique a pour but de mettre en œuvre la commande en boucle ouverte d'un moteur à courant continu (MCC). Il s'agit de contrôler la vitesse du moteur dans les deux sens de rotation en générant un signal PWM via le microcontrôleur STM32, associé à une logique de commande adaptée au circuit de puissance VN7070. Ce TP mobilise les connaissances acquises lors du TD N°1 concernant la partie matérielle, ainsi que les compétences de configuration d'une PWM développées en annexe.

Rappel du cahier des charges :

- La réalisation de ce TP s'articule en cinq étapes progressives. Les trois premières sont validées à l'oscilloscope, indépendamment du moteur, afin de vérifier le bon fonctionnement des signaux générés avant toute mise sous tension du système mécanique. Les deux dernières étapes sont ensuite validées directement sur le moteur. L'ensemble des entrées/sorties nécessaires doit être configuré de manière autonome par les étudiants, en s'inspirant des acquis du TP N°1.

La première étape consiste à initialiser le signal PWM avec une fréquence de 5 kHz et un rapport cyclique initial de 20%. La deuxième étape prévoit qu'un appui sur le bouton BP_UP augmente le rapport cyclique de 20%, avec une saturation haute bloquée à 100%. La troisième étape est symétrique : un appui sur BP_DOWN diminue le rapport cyclique de 20%, avec une saturation basse à 0%. Enfin, les quatrième et cinquième étapes concernent le contrôle du sens de rotation du moteur : un appui sur BP_RIGHT entraîne la rotation dans un sens, tandis qu'un appui sur BP_LEFT provoque la rotation dans le sens opposé.

Algorithme :

Pour répondre à ces exigences, on s'est d'abord occupé de l'initialisation :

- Configurer les broches des boutons poussoirs en entrée avec interruptions externes (EXTI).
- Configurer les broches MOT_INA et MOT_INB en sortie pour la commande du pont en H.
- Initialiser le timer TIM17 en mode PWM sur le canal 1 (PB9) avec une fréquence de 5 kHz.
- Démarrer la génération du signal PWM et fixer le rapport cyclique initial à 20% (valeur 40 sur 200).

- Définir un sens de rotation par défaut en forçant MOT_INA à l'état haut et MOT_INB à l'état bas.

Ensuite, gestion du rapport cyclique avec interruptions (BP_UP / BP_DOWN) :

Attendre un appui sur BP_UP ou BP_DOWN. Lorsqu'un appui sur BP_UP est détecté :

- Augmenter le rapport cyclique de 20% (incrément de 40).
- Si la valeur dépasse 100% (200) → la bloquer à 200.
- Appliquer immédiatement la nouvelle valeur au timer.

Lorsqu'un appui sur BP_DOWN est détecté :

- Diminuer le rapport cyclique de 20% (décrément de 40).
- Si la valeur passe en dessous de 0% → la bloquer à 0.
- Appliquer immédiatement la nouvelle valeur au timer.

Retourner en attente d'un nouvel appui.

→ L'avantage des interruptions est qu'elles permettent au microcontrôleur de modifier instantanément la vitesse du moteur dès l'appui sur un bouton, sans avoir à interroger continuellement leur état dans la boucle principale.

Ensuite, gestion du sens de rotation par scrutation (BP_RIGHT / BP_LEFT) :

Vérifier en permanence l'état des boutons BP_RIGHT et BP_LEFT dans la boucle principale. Si BP_RIGHT est appuyé :

- Mettre MOT_INA et MOT_INB à l'état haut pendant 30 ms (phase de freinage).
- Puis appliquer la combinaison INA=1, INB=0 pour faire tourner le moteur dans le sens 1.

Si BP_LEFT est appuyé :

- Mettre MOT_INA et MOT_INB à l'état haut pendant 30 ms (phase de freinage).
- Puis appliquer la combinaison INA=0, INB=1 pour faire tourner le moteur dans le sens 2.

→ Le délai de 30 ms entre les deux états logiques est une précaution nécessaire pour éviter un changement brutal de sens qui pourrait endommager le circuit de puissance VN7070 ou provoquer des à-coups mécaniques sur le moteur.

Code :

→ Voici le code réalisée ((Pour rappel vous trouverez ci-joint Le fichier PDF qui contient tout le code de ce TP TP2 , et en annexe le fichier contenant l'entièreté du code réalisé pendant ce projet avec les commentaires ajoutés)

Déclarée volatile car modifiée dans les routines d'interruption et lue dans la boucle principale, cette variable stocke le rapport cyclique courant, initialisé à 40 (soit 20%).

```
/* USER CODE BEGIN PV */  
volatile int rapport_cyclique = 40 ;  
/* USER CODE END PV */
```

Au démarrage, on lance la génération du signal PWM sur TIM17, on définit le sens de rotation par défaut (INA=1, INB=0) et on applique le rapport cyclique initial au timer.

```
/* USER CODE BEGIN 2 */  
HAL_TIM_PWM_Start(&htim17, TIM_CHANNEL_1);  
HAL_GPIO_WritePin(GPIOC, MOT_INA_Pin, GPIO_PIN_SET);  
HAL_GPIO_WritePin(GPIOC, MOT_INB_Pin, GPIO_PIN_RESET);  
__HAL_TIM_SET_COMPARE(&htim17, TIM_CHANNEL_1, rapport_cyclique);  
  
/* USER CODE END 2 */
```

En scrutation continue, si BP_RIGHT ou BP_LEFT est appuyé, on applique d'abord un freinage de 30 ms (INA=1, INB=1) avant d'imposer la combinaison logique correspondant au sens de rotation souhaité.

```

while (1)
{
    if(HAL_GPIO_ReadPin(BP_RIGHT_GPIO_Port,BP_RIGHT_Pin) == GPIO_PIN_RESET){
        HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_SET);

        HAL_Delay(30);

        HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_RESET);

    }
    if(HAL_GPIO_ReadPin(BP_LEFT_GPIO_Port,BP_LEFT_Pin) == GPIO_PIN_RESET){
        HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_SET);

        HAL_Delay(30);

        HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_SET);
    }
}
/* USER CODE END WHILE */

```

Lors d'un appui sur BP_UP, l'interruption EXTI0 incrémente le rapport cyclique de 40, le sature à 200 si nécessaire, puis applique immédiatement la nouvelle valeur au timer.

```

void EXTI0_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI0_IRQn 0 */
    rapport_cyclique = rapport_cyclique + 40;
    if(rapport_cyclique>200)
        rapport_cyclique = 200;
    __HAL_TIM_SET_COMPARE(&htim17, TIM_CHANNEL_1, rapport_cyclique);
    /* USER CODE END EXTI0_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(BP_UP_Pin);
    /* USER CODE BEGIN EXTI0_IRQn 1 */

    /* USER CODE END EXTI0_IRQn 1 */
}

```

Lors d'un appui sur BP_DOWN, l'interruption EXTI1 décrémente le rapport cyclique de 40, le bloque à 0 si nécessaire, puis met à jour le timer immédiatement.

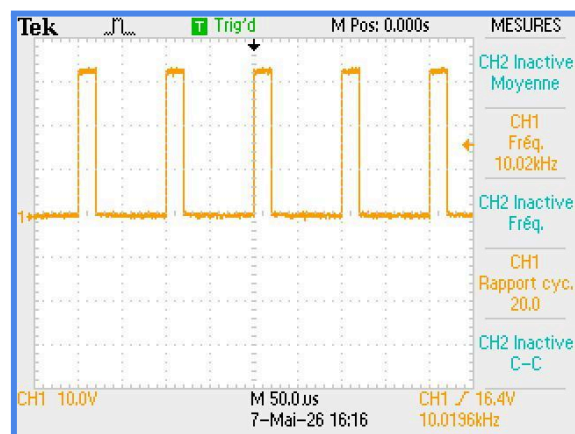
```

void EXTI1_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI1_IRQn 0 */
    rapport_cyclique = rapport_cyclique - 40;
    if(rapport_cyclique<0)
        rapport_cyclique = 0;
    __HAL_TIM_SET_COMPARE(&htim17, TIM_CHANNEL_1, rapport_cyclique);
    /* USER CODE END EXTI1_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(BP_DOWN_Pin);
    /* USER CODE BEGIN EXTI1_IRQn 1 */

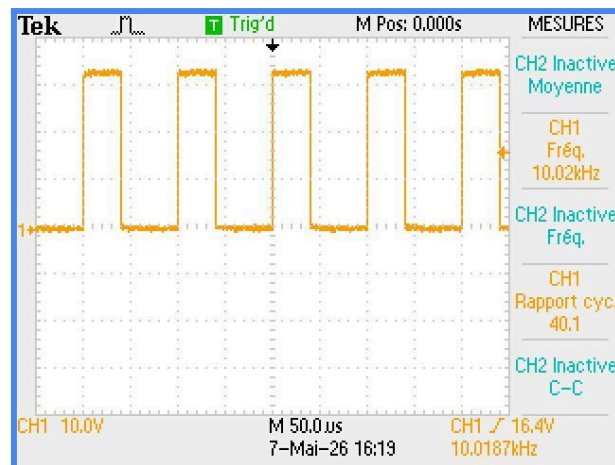
    /* USER CODE END EXTI1_IRQn 1 */
}

```

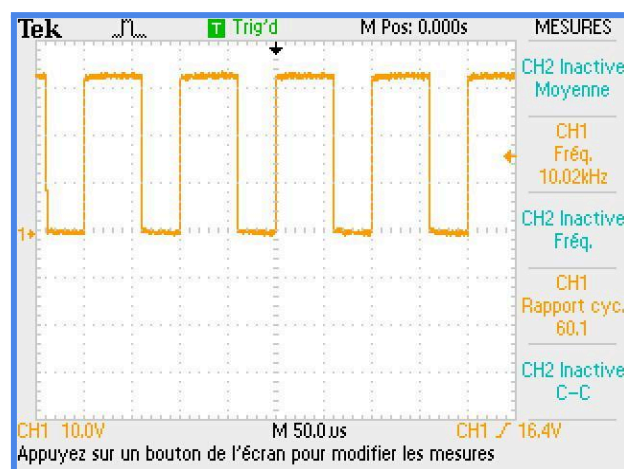
visualisation du rapport cyclique via l'oscilloscope :



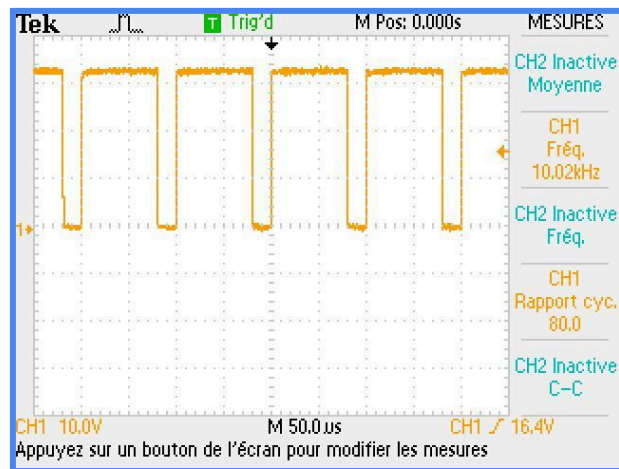
Rapport cyclique de 20% :



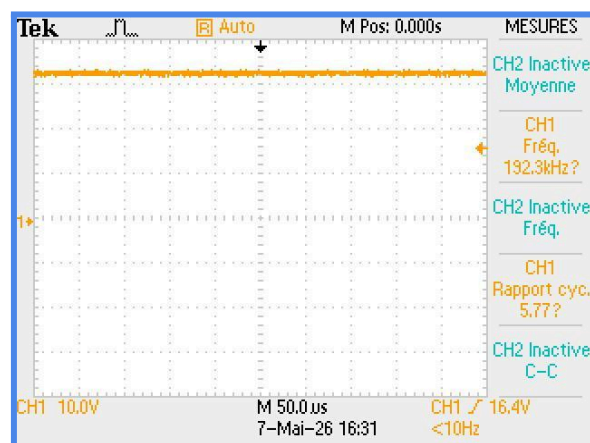
Rapport cyclique de 40% :



Rapport cyclique de 60% :



Rapport cyclique de 80% :



Rapport cyclique de 100% :

TP N°3 : Affichage d'indications sur LED reliées au bus SPI- Création d'une bibliothèque

Objectif :

- L'objectif de ce troisième TP est de piloter l'affichage des LED reliées au bus SPI afin d'indiquer l'état de fonctionnement du moteur à courant continu. Les LED permettent de visualiser le rapport cyclique appliqué au signal PWM ainsi que le sens de rotation du moteur. À travers ce TP, nous avons également découvert l'utilisation du protocole de communication SPI sur le microcontrôleur STM32 et son exploitation pour commander un système d'affichage à LED

Rappel du cahier des charges :

Le cahier des charges stipule que l'affichage des LEDs doit représenter visuellement le rapport cyclique appliqué au moteur à courant continu. Lorsque le moteur tourne dans un premier sens de rotation, les LEDs doivent s'allumer progressivement de la LSB (*Least Significant Bit*) vers la MSB (*Most Significant Bit*) en fonction du rapport cyclique : à 0 % toutes les LEDs sont éteintes, à 100 % toutes les LEDs sont allumées, et pour une valeur intermédiaire le nombre de LEDs allumées doit être proportionnel au rapport cyclique. Pour cela, une table de transcodage doit être complétée afin d'associer chaque indice à une valeur hexadécimale correspondant aux LEDs à activer, ainsi qu'une formule permettant de calculer automatiquement l'indice du tableau à partir du rapport cyclique.

Le cahier des charges impose également que, lorsque le moteur tourne dans le sens inverse, l'affichage des LEDs soit inversé : les LEDs doivent alors s'allumer progressivement de la MSB vers la LSB, toujours proportionnellement au rapport cyclique. Cette fonctionnalité peut être réalisée soit en utilisant une seconde table de transcodage adaptée au sens inverse, soit en modifiant astucieusement les indices utilisés pour accéder au tableau existant.

Algorithme :

Dans ce troisième TP, le programme réalisé au TP2 est conservé. La nouvelle fonctionnalité ajoutée consiste à afficher le rapport cyclique du moteur sur les LED pilotées par le bus SPI.

Pour répondre à ces exigences, on commence par :

- Initialiser le bus SPI1 du microcontrôleur STM32.
- Initialiser le composant MCP23S08 permettant de commander les LED.

- Créer un tableau de transcodage `leds[ ]` contenant les différentes combinaisons de LED à afficher.
- Appeler la fonction `vu_metre()` afin d'afficher l'état initial correspondant au rapport cyclique.

Ensuite, gestion de l'affichage du rapport cyclique :

- Récupérer la valeur actuelle du rapport cyclique.
- Convertir cette valeur en un indice compris entre 0 et 8.
- Utiliser cet indice pour accéder à une valeur du tableau `leds[ ]`.
- Cette valeur correspond aux LED devant être allumées.
- Envoyer cette valeur au composant MCP23S08 via le bus SPI grâce à la fonction `MCP23S08_Write()`.

Ainsi :

- lorsque le rapport cyclique augmente, davantage de LED s'allument ;
- lorsque le rapport cyclique diminue, le nombre de LED allumées diminue également.

L'affichage obtenu fonctionne donc comme un vu-mètre permettant de visualiser en temps réel la vitesse appliquée au moteur.

Code :

Voici le code ajouté (*Pour rappel vous trouverez ci-joint Le fichier PDF qui contient tout le code de ce TP TP3*, et en annexe le fichier contenant l'entièreté du code réalisé pendant ce projet avec les commentaires ajoutés)

→ Premièrement on déclare la fonction `Vu_metre()` dans le `/* USER CODE BEGIN PFP */`

```
53  /* USER CODE BEGIN PFP */
54  void MCP23S08_Init(void);
55  void MCP23S08_Write(uint8_t data);
56  void vu_metre(void);
57  /* USER CODE END PFP */
--
```

On écrira le contenu de cette fonction dans l'endroit prévu à l'usage c'est à dire dans le `/* USER CODE BEGIN 4 */`

```
/* USER CODE BEGIN 4 */
void MCP23S08_Init(void) {
    uint8_t tab[3] = {0X40,0,0}; // création d'un tableau de 3 uint : {0X40 ,0,0}
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port,SPI_CS_Pin,GPIO_PIN_RESET); // allumer le GPIO SPI_CS
    HAL_SPI_Transmit(&hspi1, tab,3,1000); //envoie le tableau en SPI
    // attendre que la ligne soit libre
    while(hspi1.State == HAL_SPI_STATE_BUSY){}
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port,SPI_CS_Pin,GPIO_PIN_SET); // eteindre le GPIO SPI_CS
}

void MCP23S08_Write(uint8_t data) {
    uint8_t tab[3] = {0X40,9,data}; //création d'un tableau de 3 uint : {0X40, 9, data}
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port,SPI_CS_Pin,GPIO_PIN_RESET); //allumer le GPIO SPI_CS
    HAL_SPI_Transmit(&hspi1, tab,3,1000); //envoie le tableau en SPI
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port,SPI_CS_Pin,GPIO_PIN_SET); //eteindre le GPIO SPI_CS
}
```

→ On remarque par la suite que la partie SPI ajoutée au TP3 ne fonctionne pas “en boucle” dans le while.

Elle fonctionne grâce aux deux fonctions :

- MCP23S08_Init();
- vu_metre();

TP N°4 : Affichage d'indications sur écran OLED relié au bus I2C – Utilisation d'une bibliothèque externe

Objectif :

L'objectif de ce quatrième TP est d'afficher sur l'écran OLED (SSD1306, 128×64 pixels, relié au bus I2C) des informations dynamiques reflétant l'état du système : sens de rotation du moteur, fréquence du signal PWM et rapport cyclique courant. Ce TP ne requiert pas d'implémenter le protocole I2C manuellement ; la bibliothèque `ssd1306` fournie abstrait la formation des trames I2C et expose des fonctions de haut niveau pour l'affichage de texte.

Rappel du cahier des charges :

Le cahier des charges impose trois phases d'affichage successives, chaque appui sur un bouton poussoir mettant à jour l'écran dynamiquement :

- Phase 1 — Message d'accueil affiché pendant 2 secondes au démarrage.
- Phase 2 — Affichage des correspondances entre les boutons poussoirs (BP_UP, BP_DOWN, BP_RIGHT, BP_LEFT) et leurs actions, pendant 2 secondes.
- Phase 3 — Affichage dynamique sur trois lignes : la première indique le sens de rotation (“---” si non défini, “>>>” pour le sens 1, “<<<” pour le sens 2) ; la deuxième affiche la fréquence du signal PWM ; la troisième indique le rapport cyclique en pourcentage.

Algorithme :

Phase d'initialisation et messages fixes :

- Appeler `ssd1306_Init()` pour initialiser l'écran OLED via le bus I2C1.
- Effacer l'écran (`ssd1306_Fill(Black)`), positionner le curseur et écrire le message d'accueil (`ssd1306_WriteString`).
- Mettre à jour l'affichage (`ssd1306_UpdateScreen`) puis attendre 2 secondes (`HAL_Delay(2000)`).
- Afficher de la même manière le récapitulatif des boutons poussoirs, puis attendre à nouveau 2 secondes.

Mise à jour dynamique de l'affichage dans la boucle principale :

- À chaque modification d'état (appui sur un BP), effacer l'écran.
- Ligne 1 : tester la variable de sens de rotation et afficher le symbole correspondant via `ssd1306_WriteString`.

- Ligne 2 : afficher la fréquence du signal PWM (valeur calculée à partir des paramètres du timer TIM17 : prescaler et période).
- Ligne 3 : convertir le rapport cyclique (variable globale volatile) en chaîne de caractères avec `sprintf`, puis afficher via `ssd1306_WriteString`.
- Appeler `ssd1306_UpdateScreen` pour envoyer le buffer vers l'écran via I2C.

→ L'utilisation de la bibliothèque `ssd1306` découple entièrement la logique applicative de la formation des trames I2C. La fonction `ssd1306_UpdateScreen` déclenche l'envoi du buffer interne par un unique appel `HAL_I2C_Master_Transmit`, ce qui garantit une mise à jour atomique de l'affichage.

Code :

→ Voici le code réalisé (pour rappel, vous trouverez en annexe l'entièreté du code du projet avec les commentaires) :

Initialisation de l'écran OLED et affichage du message d'accueil :

```
ssd1306_Init();
```

Mise à jour dynamique de l'affichage — effacement de l'écran, affichage du sens de rotation sur la ligne 1, de la fréquence PWM sur la ligne 2 et du rapport cyclique sur la ligne 3, suivi d'un `ssd1306_UpdateScreen` :

```
if (modif) {
    char txt[30];
    ssd1306_Fill(Black);
    ssd1306_SetCursor(2, 4);
    ssd1306_WriteString("01 OLED", Font_11x18, White);
    ssd1306_SetCursor(2, 24);
    sprintf(txt, "RC:%d%%", rapport_cyclique / 2);
    printf("RC:%d%%\n\r", rapport_cyclique / 2);
    ssd1306_WriteString(txt, Font_7x10, White);
    ssd1306_SetCursor(2, 36);
    sprintf(txt, "Sens %s", sens ? ">>>" : "<<<");
    ssd1306_WriteString(txt, Font_7x10, White);
    HAL_Delay(300);
    HAL_ADC_PollForConversion(&hadc1, 100);
    val_adc = HAL_ADC_GetValue(&hadc1);
    vitesse = val_adc / (k1 * k2 * k3);
    ssd1306_SetCursor(2, 48);
    sprintf(txt, "v:%d tr/min", vitesse);
    ssd1306_WriteString(txt, Font_7x10, White);
    ssd1306_UpdateScreen();
    modif = 0;
}
```

Voici le code ajouté (*Pour rappel vous trouverez ci-joint Le fichier PDF qui contient tout le code de ce TP TP4 , et en annexe le fichier contenant l'entièreté du code réalisé pendant ce projet avec les commentaires ajoutés*)

TP N°5 : Mesure de la vitesse de rotation du moteur – ADC

Objectif :

L'objectif de ce TP est de réaliser l'acquisition analogique de la tension fournie par la génératrice, cette tension étant représentative de la vitesse de rotation du moteur à courant continu. Les données acquises sont ensuite converties numériquement puis affichées sur l'écran OLED, avec la valeur mesurée ainsi que la vitesse de rotation du moteur exprimée en tours par minute (tr/min).

Rappel du cahier des charges :

Le cahier des charges de ce TP est divisé en deux parties complémentaires. La première constitue une phase de prise en main de l'acquisition analogique sur STM32, tandis que la seconde permet de répondre directement aux objectifs du projet.

La première étape consiste à configurer l'entrée analogique reliée au potentiomètre en s'appuyant sur l'annexe VII. Le programme doit ensuite réaliser une conversion analogique-numérique à l'aide de l'ADC, récupérer la valeur numérique brute obtenue puis l'afficher sur l'écran OLED. À partir de cette valeur, il faut également calculer et afficher la tension correspondante en tenant compte de la résolution de l'ADC (12 bits) et de la tension de référence de 3,3 V.

Cette partie étant pas en rapport directe avec le projet du moteur à courant continue, nous avons décidé de ne pas y consacrer beaucoup de temps, étant donné que cette partie nous a juste permis de comprendre le fonctionnement de L'ADC dans cet environnement

La seconde étape, qui est la plus importante, consiste à configurer l'entrée analogique connectée à la génératrice utilisée pour mesurer la vitesse du moteur à courant continu. Le programme doit alors effectuer l'acquisition de cette tension, afficher la valeur numérique convertit ainsi que calculer et afficher la vitesse de rotation du moteur en tours par minute (tr/min) sur l'écran OLED.

Algorithme :

Dans ce cinquième TP, le programme développé lors des TP précédents est conservé afin de maintenir :

- la commande PWM du moteur ;
- la gestion du sens de rotation ;
- l'affichage des informations sur l'écran OLED ;
- ainsi que l'affichage du rapport cyclique sur les LED via le bus SPI.

La nouvelle fonctionnalité ajoutée consiste à acquérir la tension issue de la génératrice grâce à l'ADC afin de calculer et d'afficher la vitesse de rotation du moteur en tr/min sur l'écran OLED.

Initialisation de l'ADC

Pour répondre à ces exigences, on commence par :

- Initialiser l'ADC1 du microcontrôleur STM32.
 - Démarrer les conversions analogique-numérique
 - Définir les constantes k1, k2 et k3 permettant de convertir la valeur numérique de l'ADC en vitesse de rotation du moteur.
-

Acquisition de la tension de la génératrice

Dans la boucle principale :

- Attendre qu'une conversion ADC soit terminée grâce à `HAL_ADC_PollForConversion()`.
- Lire la valeur numérique convertie avec `HAL_ADC_GetValue()`.
- Stocker cette valeur dans la variable `val_adc`.

La tension mesurée correspond à l'image de la vitesse de rotation du moteur.

Calcul de la vitesse du moteur

Une fois la valeur ADC récupérée :

- Calculer la vitesse de rotation du moteur à l'aide de la relation :

$$\text{vitesse} = \text{val_adc} / (k1 * k2 * k3);$$

où :

- `k1` correspond au gain lié à la génératrice

→ Comme vu en TD, $k1 = 24/3750$ avec 24 la tension maximale et 3750 la vitesse maximale en tour par minute

- $k2$ correspond au pont diviseur de tension :

→ Comme vu en TD, $k2 = 1,6 / (1,6 + 10)$ qui représente directement un pont diviseur de tension résistif et qui sert à convertir la tension en 3,3 V.

- $k3$ correspond au facteur de conversion de l'ADC 12 bits.

→ Comme vu en TD, $k3 = 4095 / 3,3$ avec 4095 qui représente le nombre de valeurs maximale sur 12 bits 2^{12}

Le résultat obtenu correspond à la vitesse de rotation du moteur en tours par minute.

Mise à jour de l'affichage OLED

Après le calcul :

- Effacer l'écran OLED.
- Afficher :
 - le rapport cyclique ;
 - le sens de rotation du moteur ;
 - la vitesse de rotation calculée en tr/min.
- Actualiser l'écran

Ainsi, l'utilisateur peut visualiser en temps réel les paramètres de fonctionnement du moteur directement sur l'écran OLED.

Code :

Voici le code ajouté (*Pour rappel vous trouverez ci-joint Le fichier PDF qui contient tout le code de ce TP TP5, et en annexe le fichier contenant l'entièreté du code réalisé pendant ce projet avec les commentaires ajoutés*)

Premièrement on initialise les différentes variables:

val_adc qui représente la valeur de l'ADC, la vitesse qui sera incrémenté directement par la valeur de l'ADC, et ensuite, nos trois facteurs initialisé en float

```
volatile int val_adc;
volatile int vitesse;
volatile float k1 = 24.0F/3750.0F;
volatile float k2 = 1.6F/(1.6F+10.0F);
volatile float k3 = (4095.0F/3.3F);
```

Ensuite, on démarre l'ADC dans la partie */*USER CODE BEGIN 2 */*

```
//ADC
HAL_ADC_Start(&hadc1);
```

On attend comme précisé dans l'algorithme, qu'une conversion soit effectuée, puis on lit la valeur de l'ADC afin de la stocker dans la variable prévu à cet usage

Enfin On calcule la vitesse du rotation du moteur à l'aide du rapport de la valeur de l'ADC et des trois coefficients

La mise à jour de l'écran OLED a été réalisé à la fin de la boucle

→ il est important de mettre ces lignes dans la condition `if(modif) {`

car la variable "modif", comme précisé dans le TP précédent prend la valeur de 1 quand on appuie sur un des boutons contrôlant directement le moteur à courant continue


```

}
if (modif){
    char txt[30];
    ssd1306_Fill(Black);
    ssd1306_SetCursor(2,4);
    ssd1306_WriteString("01 OLED", Font_11x18, White);
    ssd1306_SetCursor(2,24);
    sprintf(txt,"RC:%d%%",rapport_cyclique/2);
    ssd1306_WriteString(txt, Font_7x10, White);
    ssd1306_SetCursor(2,36);
    sprintf(txt, "Sens %s", sens?">>>":"<<<");
    ssd1306_WriteString(txt, Font_7x10, White);
    HAL_Delay(300);
    HAL_ADC_PollForConversion(&hadc1,100);

    val_adc = HAL_ADC_GetValue(&hadc1);
    vitesse = val_adc/(k1*k2*k3);

    ssd1306_SetCursor(2,48);
    sprintf(txt,"v:%d tr/min",vitesse);
    ssd1306_WriteString(txt, Font_7x10, White);
    ssd1306_UpdateScreen();

    modif = 0;
}

```

TP N°6 : Transmission et réception de données sur le port série asynchrone – UART

Objectif :

L'objectif de ce sixième et dernier TP est d'établir une communication série asynchrone bidirectionnelle entre la carte STM32 et un PC via l'interface USART2 (broches PA2/PA3), configurée à 19 200 bits/s, 8 bits de données, sans parité, 1 bit de stop. L'application doit permettre de piloter le MCC depuis le terminal série Termite en envoyant des caractères de commande, et de retransmettre en temps réel la vitesse de rotation vers le PC.

Rappel du cahier des charges :

Le cahier des charges se décompose en quatre étapes progressives :

- Étape 1 — Configuration du port USART2 avec les paramètres spécifiés, redirection de la fonction printf vers le port série (via __io_putchar).
- Étape 2 — Vérification de la liaison en affichant sur l'écran OLED les caractères reçus depuis Termite.

- Étape 3 — Implémentation de la logique de commande par caractère : la réception de 'U' déclenche l'action BP_UP, 'D' l'action BP_DOWN, 'R' l'action BP_RIGHT (sens 1), 'L' l'action BP_LEFT (sens 2).
- Étape 4 — Transmission de la vitesse de rotation à chaque appui sur un bouton poussoir, envoyée toutes les millisecondes via printf.

Les fonctions de communication sont utilisées avec un TimeOut de 1 ms : si aucune donnée n'est reçue dans ce délai, la boucle principale reprend son exécution sans blocage.

Algorithme :

Initialisation de l'USART2 :

- CubeMX configure automatiquement les broches PA2 (USART2_TX) et PA3 (USART2_RX) en fonction alternative USART2.
- Rediriger printf en implémentant la fonction `__io_putchar(int ch)` qui appelle `HAL_UART_Transmit` avec un TimeOut de 1 ms.

Réception et décodage des commandes :

- Dans la boucle principale, appeler `HAL_UART_Receive` avec un buffer d'un octet et un TimeOut de 1 ms.
- Tester le caractère reçu : si 'U' → incrémenter le rapport cyclique ; si 'D' → décrémenter ; si 'R' → appliquer INA=1, INB=0 après freinage de 30 ms ; si 'L' → appliquer INA=0, INB=1 après freinage.
- Mettre à jour le timer TIM17 et l'affichage OLED/vu-mètre après chaque commande reçue.

Transmission de la vitesse de rotation :

- Lorsqu'un bouton poussoir est actionné, appeler printf pour transmettre la vitesse de rotation calculée à partir de la valeur ADC.
- Le TimeOut de 1 ms sur `HAL_UART_Transmit` garantit que la transmission n'est pas bloquante : si le buffer d'émission n'est pas disponible, la boucle reprend immédiatement.

→ L'utilisation d'un TimeOut de 1 ms sur `HAL_UART_Receive` et `HAL_UART_Transmit` est une précaution essentielle dans un contexte temps réel : elle évite tout blocage de la boucle principale en l'absence de données sur la liaison série, garantissant ainsi la continuité du contrôle moteur.

Code :

→ Voici le code réalisé (pour rappel, vous trouverez en annexe l'entièreté du code du projet avec les commentaires) :

Redirection de printf vers USART2 — implémentation de `__io_putchar` appelant `HAL_UART_Transmit` avec un TimeOut de 1 ms :

```
32 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
33 #define GETCHAR_PROTOTYPE int fgetc(FILE *f)
```

```

77  PUTCHAR_PROTOTYPE {
78      HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 1000);
79      return ch;
80  }
81  GETCHAR_PROTOTYPE {
82      int ch;
83      HAL_UART_Receive(&huart2, (uint8_t *)&ch, 1, 1000);
84      return ch;
85  }

```

Réception d'un caractère depuis Termite et décodage de la commande — appel HAL_UART_Receive avec TimeOut 1 ms, puis test sur le caractère reçu {U, D, R, L} et application de l'action moteur correspondante :

```

143      uint8_t ch = 0 ;
144      HAL_UART_Receive(&huart2, &ch, 1, 1);

169      if (ch == 'U') {
170          EXTIO_IRQHandler();
171      }
172      if( ch == 'D') {
173          EXTII_IRQHandler();
174      }

145      if(HAL_GPIO_ReadPin(BP_RIGHT_GPIO_Port, BP_RIGHT_Pin) == GPIO_PIN_RESET || ch== 'R'){
146          HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_SET);
147          HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_SET);
148
149          HAL_Delay(30);
150
151          HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_SET);
152          HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_RESET);
153
154          sens = 0;
155          modif = 1;
156      }
157      if(HAL_GPIO_ReadPin(BP_LEFT_GPIO_Port, BP_LEFT_Pin) == GPIO_PIN_RESET || ch == 'L') {
158          HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_SET);
159          HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_SET);
160
161          HAL_Delay(30);
162
163          HAL_GPIO_WritePin(MOT_INA_GPIO_Port, MOT_INA_Pin, GPIO_PIN_RESET);
164          HAL_GPIO_WritePin(MOT_INB_GPIO_Port, MOT_INB_Pin, GPIO_PIN_SET);
165
166          sens = 1;
167          modif = 1;
168      }

```

Transmission de la vitesse de rotation vers le PC — appel printf formatant la valeur de vitesse en tr/min et l'envoyant sur USART2 :

```

175     if (modif){
176         char txt[30];
177         ssdl306_Fill(Black);
178         ssdl306_SetCursor(2,4);
179         ssdl306_WriteString("01 OLED", Font_11x18, White);
180         ssdl306_SetCursor(2,24);
181         sprintf(txt, "RC:%d%%", rapport_cyclique/2);
182         printf("RC:%d%%\n\r", rapport_cyclique/2);
183         ssdl306_WriteString(txt, Font_7x10, White);
184         ssdl306_SetCursor(2,36);
185         sprintf(txt, "Sens %s", sens?">>>":"<<<");
186         ssdl306_WriteString(txt, Font_7x10, White);
187         HAL_Delay(300);
188         HAL_ADC_PollForConversion(&hadcl, 100);
189
190         val_adc = HAL_ADC_GetValue(&hadcl);
191         vitesse = val_adc/(k1*k2*k3);
192
193         ssdl306_SetCursor(2,48);
194         sprintf(txt, "v:%d tr/min", vitesse);
195         ssdl306_WriteString(txt, Font_7x10, White);
196         ssdl306_UpdateScreen();
197
198
199         modif = 0;

```

Voici le code ajouté (Pour rappel vous trouverez ci-joint Le fichier PDF qui contient tout le code de ce TP TP6 , et en annexe le fichier contenant l'entièreté du code réalisé pendant ce projet avec les commentaires ajoutés)

4. Conclusion générale

Au cours de ces différents travaux pratiques, nous avons progressivement découvert les principales fonctionnalités du microcontrôleur STM32 ainsi que les outils de développement associés. Les différentes séances nous ont permis de mettre en œuvre plusieurs périphériques matériels essentiels des systèmes embarqués, tels que les GPIO, les interruptions externes, les timers PWM, les communications SPI et I2C, l'ADC ainsi que l'UART.

L'ensemble des TP réalisés a conduit au développement progressif d'une application complète de pilotage et de supervision d'un moteur à courant continu. À partir d'une commande simple de LED lors du premier TP, le projet a ensuite évolué vers :

- la commande en vitesse du moteur par PWM ;
- la gestion du sens de rotation ;
- l'affichage du rapport cyclique grâce à un vu-mètre à LED piloté en SPI ;
- l'affichage d'informations sur un écran OLED via le protocole I2C ;
- la mesure de la vitesse réelle du moteur à l'aide de l'ADC ;

- ainsi que la communication avec un ordinateur grâce à l'UART et le logiciel PUTTY .

Ces travaux nous ont également permis de mieux comprendre l'architecture d'un système embarqué complet, en associant la partie logicielle développée en langage C avec les différents périphériques matériels configurés sous STM32CubeMX. Nous avons aussi appris à structurer un programme embarqué en utilisant les bibliothèques HAL fournies par STMicroelectronics.

Enfin, ce projet nous a permis de développer des compétences importantes en programmation embarquée, en débogage, ainsi qu'en intégration progressive de plusieurs fonctionnalités dans une même application temps réel.

5. Annexes

→ Vous trouverez ci-joint le dossier des annexes comprenant :

- La vidéo illustrant le résultat final du projet
- L'intégralité du code source développé durant les séances de TP, dûment commenté.
- Les visualisations via l'oscilloscope

 Annexes